Instituto Tecnológico de Culiacán

Ingeniería en Sistemas Computacionales

"Inteligencia Artificial"

Tarea:

Reflexion. Tabla Comparativa.

Alumno:

Monreal Navarro Diego Alan

Docente:

Jose Mario Rios Felix

Culiacán, Sinaloa, 3 de marzo de 2026

Algoritmo	Definición	Cómo funciona (8-puzzle)	Ventajas	Desventajas	Cuando conviene usarlo	¿Es Óptimo?
Búsqueda en Anchura (BFS)	Este explora el espacio de estados por niveles, ósea, primero los estados a 1 movimiento, luego 2, luego 3, y así constantemente hasta su objetivo.	Usa una cola (FIFO): saca el primer estado, luego genera sus vecinos y luego los mete al final. Con como "visitados", ya que esto evita repetir los estados.	-Encuentra la solución con menos movimientos si cada movimiento cuesta 1. -Es muy fácil de implementar.	-Consume mucha memoria -Tiempo alto cuando la profundidad de la solución es grande.	Cuando quieres la solución más corta en número de movimientos y el problema no es demasiado grande, entonces tienes memoria suficiente para usar este algoritmo	Sí, si cada movimiento tiene costo uniforme
Búsqueda en Profundidad (DFS)	Este lo que hace es basicamente explorar lo más profundo posible antes de retroceder.	Usa una pila (LIFO), pero sigue una rama de movimientos hasta que ya no pueda o convenga, luego hace backtracking, usualmente se usa con "visitados" o con límite de profundidad para no ciclar.	-Muy poca memoria comparada con BFS/UCS. -Puede encontrar una solución "rápido" si por suerte cae en una buena rama. -Implementación sencilla.	-Puede irse por ramas largas e inútiles. -Puede tardar mucho si no encuentra o cae en una solución. -No tiene control de ciclos o de profundidad, y normalmente puede ciclar.	-Cuando la memoria es el principal límite, o solo necesitas una solución, puede ser conveniente usarlo, también es útil como base de Depth-Limited / IDDFS.	No, este si no garantiza la mejor ni en costo ni en movimientos.
Costo Uniforme (UCS)	Expande siempre el nodo con menor costo acumulado (g(n)).	En este caso, este usa una cola de prioridad ordenada por el costo total desde el inicio, y va eligiendo el estado con menor costo y expande.	-Óptima si los costos son no negativos. -Completa bajo condiciones normales (costos ≥ 0). -Maneja bien costos distintos por movimiento.	-Puede ser lenta si hay muchos estados con costo parecido. -También puede consumir mucha memoria	Cuando los movimientos no cuestan igual (o quieres minimizar un costo real, no solo número de movimientos). Si todos cuestan 1, normalmente sería mejor usar BFS.	Sí, si todos los costos son ≥ 0

Cuál es la diferencia entre A* y IDA*?

A* y IDA* usan la misma idea de evaluación que es esta: $f(n) = g(n) + h(n)$(costo acumulado + heurística), pero solamente cambian en cómo recorren y qué tanto guardan, por ejemplo:

- A* mantiene una lista abierta en una cola de prioridad y siempre expande el nodo con menor f, y por eso suele ser muy eficiente en tiempo, pero su gran problema es la memoria, porque este va guardando muchísimos estados
- IDA* (Iterative Deepening A* no usa una cola de prioridad grande, sino que hace una búsqueda tipo profundidad, pero con un límite de f, ósea, primero permite solo nodos con $f \leq L$, y si no encuentra solución, sube el límite al siguiente valor necesario y repite. Esto hace que use muy poca memoria, pero a cambio repite expansiones en cada iteración, así que puede ser un poco mas lento.